

One-More security assumptions

i This report was written as part of the 2025 edition of the self-study PhD course “Cryptographic Proof Techniques” by Prof. Elena Pagnin at Chalmers University of Technology, Göteborg, Sweden.

1 Introduction: on provable security

ADVERSARIAL MODEL. The security of any cryptographic scheme Π is usually established in terms of the capabilities and goal of an attacker $\mathcal{A}$. For instance, a standard security notion for signature schemes Π_{Sign} is *existential unforgeability under chosen message attack* (EUF-CMA), where $\mathcal{A}$ has access to a signing oracle $\mathcal{O}_{\text{Sign}}$ and aims at providing a valid message-signature pair (m^*, σ^*) for a new message m^* . Further, adopting models (e.g. *standard model*, *generic group model*, *algebraic group model*) captures the algebraic structures the adversary $\mathcal{A}$ is allowed to work with.

SECURITY REDUCTIONS. Consider security claims for signature schemes Π_{Sign} , usually of the form

“If challenge is hard, scheme Π_{Sign} is EUF-CMA secure”

where challenge is a computational problem believed to be hard to solve. In game-based proofs, such claims are proved via the contrapositive proposition:

“If Π_{Sign} is not EUF-CMA secure, then challenge is not hard.”

In other words we show that, with access to a PPT adversary $\mathcal{A}$ that breaks the EUF-CMA security of Π_{Sign} with non-negligible probability $\epsilon_{\mathcal{A}}$, we are able to exhibit a PPT extractor $\mathcal{E}$ able to simulate an execution of Π_{Sign} to break challenge with non-negligible probability $\epsilon_{\mathcal{E}}$. This success probability, compared to the best algorithms known to attack challenge, allows them to set the parameters of the scheme, so we want the reduction to be as tight as possible.

2 The One-More RSA Inversion assumption

ONE-MORE ONE-WAY FUNCTIONS. *One-More forgeries* were first formalised by Pointcheval and Stern in [1]. Intuitively, they wish to capture the attacker capability of querying an oracle before the challenge even starts. This is done by allowing t interactions with some oracle and then asking to produce “one more” forgery, the $(t + 1)$ -th. This later inspired the formal definition of *one-more one-way function*, introduced by Bellare et al. in [2] as an interactive game between a challenger $\mathcal{C}$ and adversary $\mathcal{A}$. The challenger $\mathcal{C}$ generates $t + 1$ inversion challenges. The adversary $\mathcal{A}$ can query the inversion oracle $\mathcal{O}^{\text{inv-}f}$ for f in Figure 1, and it wins if it can invert all t challenges while performing less than t queries to the inversion oracle.

Definition 1 (One-more one-way function [2]). A function f is one-more one-way if it can be computed in polynomial time with respect to the size of its output and no PPT adversary $\mathcal{A}$ has non-negligible probability of winning the one-more inversion game.

$\mathcal{O}^{\text{inv-}f}(x)$ if $y \notin f(X)$ return $\perp$ $x \leftarrow f^{-1}(y)$ $q \leftarrow q + 1$ return x	$\mathcal{O}^{\text{chal-}f}()$ $x \leftarrow \\$ X$ $y \leftarrow f(x)$ $q \leftarrow q + 1$ return y
---	--

Figure 1: On the left, the inversion oracle in the one-more one-way inversion game. On the right, the challenge oracle in *alternative* one-more one-way inversion games.

THREE FLAVOURS OF RSA INVERSION. Bellare et al. adapt the notion of one-more one-way functions to the RSA function. Consider the classical RSA assumption, renamed STI-RSA in [2], where a parameter generation algorithm RsaGen with input the security parameter λ returns $N = pq$, the encryption exponent e and the decryption exponent d .

Assumption 2 (RSA Single-Target Inversion [2]). Let RsaGen be the RSA parameter generation algorithm, and let game $\text{Game}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-STI}}$ be as defined in Figure 2. We say that the RSA Single-Target Inversion (RSA-STI) problem is hard if, for any PPT adversary $\mathcal{A}$ playing $\text{Game}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-STI}}$, it holds

$$\text{Adv}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-STI}}(\lambda) := \mathbb{P} \left[\text{Game}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-STI}}(\lambda) = 1 \right] = \text{negl}(\lambda) \quad (1)$$

This can be generalised by generating $m + 1$ challenges and allowing the adversary to query an RSA inversion oracle (Figure 1) at most m times.

Assumption 3 (RSA Known-Target Inversion [2]). Let RsaGen be the RSA parameter generation algorithm and $m = m(\lambda)$ for λ security parameter. Let game $\text{Game}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-KTI}}$ be as defined in Figure 2. We say that the RSA Known-Target Inversion (RSA-KTI _{m}) problem is hard if, for any PPT adversary $\mathcal{A}$ playing $\text{Game}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-KTI}}$, it holds

$$\text{Adv}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-KTI}}(m, \lambda) := \mathbb{P} \left[\text{Game}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-KTI}}(m, \lambda) = 1 \right] = \text{negl}(\lambda) \quad (2)$$

A further generalisation allows the adversary $\mathcal{A}$ to choose which $m + 1$ out of n challenges it wishes to invert, with $m < n$. Due to this, we call it “chosen-target”. The adversary $\mathcal{A}$ will also provide an injective map $\pi : \{1, \dots, m + 1\} \rightarrow \{1, \dots, n\}$ to denote which $m + 1$ challenges it chose.

Assumption 4 (RSA Chosen-Target inversion [2]). Let RsaGen be the RSA parameter generation algorithm, $m = m(\lambda)$ and $n = n(\lambda)$ for λ security parameter. Let game $\text{Game}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-CTI}}$ be as defined in Figure 2. We say that the RSA (n, m) -Chosen-Target Inversion (RSA-CTI _{n, m}) problem is hard if, for any PPT adversary $\mathcal{A}$ playing $\text{Game}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-CTI}}$, it holds

$$\text{Adv}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-CTI}}(n, m, \lambda) := \mathbb{P} \left[\text{Game}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-CTI}}(n, m, \lambda) = 1 \right] = \text{negl}(\lambda) \quad (3)$$

Game $^{\text{RSA-STI}}_{\mathcal{A}, \text{RsaGen}}(\lambda)$ $(N, e, d) \leftarrow \text{RsaGen}(1^\lambda)$ $y \leftarrow \$\mathbb{Z}_N;$ $x \leftarrow \mathcal{A}(N, e, d, y)$ return 1 if $x^e = y$ return 0	Game $^{\text{RSA-KTI}}_{\mathcal{A}, \text{RsaGen}}(m, \lambda)$ $(N, e, d) \leftarrow \text{RsaGen}(1^\lambda)$ for $i = 1, \dots, m + 1$ $y_i \leftarrow \$\mathbb{Z}_N;$ $\vec{y} \leftarrow (y_1, \dots, y_{m+1})$ $\vec{x} \leftarrow \mathcal{A}^{\text{O}^{\text{inv-RSA}}}(N, e, d, \vec{y})$ return 1 if $\forall i \ x_i^e = y_i$ and $q \leq m$ return 0	Game $^{\text{RSA-CTI}}_{\mathcal{A}, \text{RsaGen}}(n, m, \lambda)$ $(N, e, d) \leftarrow \text{RsaGen}(1^\lambda)$ for $i = 1, \dots, n$ $y_i \leftarrow \$\mathbb{Z}_N;$ $\vec{y} \leftarrow (y_1, \dots, y_n)$ $(x_1, \dots, x_{m+1}, \pi) \leftarrow \mathcal{A}^{\text{O}^{\text{inv-RSA}}}(N, e, d, \vec{y})$ return 1 if $\forall i \ x_i^e = y_{\pi(i)}$ and $q \leq m$ return 0
---	---	--

Figure 2: Games for the three variants of the RSA assumption: *Single-Target Inversion*, *Known-Target Inversion*, *Chosen-Target Inversion*. The main differences between the one-more variants, RSA-KTI and RSA-CTI, are highlighted in grey.

Lemma 5 ([2]). *RSA-CTI $_{n,m}$ is hard if and only if RSA-KTI $_m$ is hard.*

Sketch of proof. Consider a reduction $\mathcal{B}$ playing the RSA-CTI $_{n,m}$ game on input $y_1, \dots, y_n$, then $\mathcal{B}$ can just run a PPT adversary $\mathcal{A}$ winning the RSA-KTI $_m$ game on the first $m + 1$ points and return the injective map $\pi(i) = i$ for $i = 1, \dots, m + 1$.

The other side is harder, we only provide the main idea. Consider a reduction $\mathcal{B}$ playing the RSA-KTI $_m$ game on input the target points $y_1, \dots, y_{m+1}$. First, $\mathcal{B}$ creates randomised targets $y'_1, \dots, y'_n$, where $y'_i = y'_i(y_1, \dots, y_{m+1})$, with the property that inverting them allows to invert the original targets. This is called “reversible embedding”, and requires multiplying the y_i elevated by a known exponent, plus a random $(m + 2)$ -th element of known inverse to ensure the y'_i are uniformly and independently distributed. Then the reduction $\mathcal{B}$ can run some PPT RSA-KTI $_{n,m}$ adversary $\mathcal{A}$, making at most m oracle queries and returning $x'_i = y'^{d'}_i$. The reduction builds an exponent matrix (a la index calculus), which is invertible if its determinant is non-zero, otherwise $\mathcal{B}$ returns $\perp$. $\square$

ALTERNATIVE FORMULATIONS: CHALLENGE ORACLE. Finally, Bellare et al. define the *RSA Alternative KTI* (RSA-AKTI $_m$) and the *RSA Alternative CTI* (RSA-ACTI $_{n,m}$) assumptions. Adversaries $\mathcal{A}$ are not given their challenge points by the challenger $\mathcal{C}$, but they can access a challenge oracle $\mathcal{O}^{\text{chal-RSA}}$ for the RSA function (Figure 1) to dynamically generate them. They prove that the original and alternative formulations are equivalent: RSA-AKTI $_m$ is hard if and only if RSA-KTI $_m$ is hard, and RSA-ACTI $_{n,m}$ is hard if and only if RSA-CTI $_{n,m}$ is hard.

3 The One-More Discrete Logarithm assumption

GENERIC GROUP MODEL. The *generic group model* (GGM) acts as a sanity check for group-based games, ensuring that the structure of the algorithm itself does not leak information. Adversaries $\mathcal{A}$ only have access to elements of a group $(\mathbb{G}, \cdot)$ as images of an injective handle function $\Xi : \mathbb{G} \rightarrow S$ mapping onto some set S without group structure, and to an implicit group computation oracle $\mathcal{O}_{\mathbb{G} \times \mathbb{G}}$ to perform the operation $(\Xi(g), \Xi(h)) \mapsto \Xi(gh)$.

DISCRETE LOGARITHM. In [3], Diffie and Hellman ponder the possibility of basing what will be known as *public key cryptosystems* on problems that are computationally hard to invert. ElGamal will formalise the Discrete Logarithm assumption in [4].

Assumption 6 (Discrete Logarithm [3, 4]). *Let GrGen be a group generation algorithm, and let game $\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{DL}}$ be as defined in Figure 3. We say that the Discrete logarithm (DL) problem is hard if, for any PPT adversary $\mathcal{A}$ playing $\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{DL}}$, it holds*

$$\text{Adv}_{\mathcal{A}, \text{GrGen}}^{\text{DL}}(\lambda) := \mathbb{P} \left[\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{DL}}(\lambda) = 1 \right] = \text{negl}(\lambda) \quad (4)$$

$\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{DL}}(\lambda)$ $(\mathbb{G}, g, p) \leftarrow \text{GrGen}(1^\lambda)$ $x \leftarrow \mathbb{Z}_p, h \leftarrow g^x$ $y \leftarrow \mathcal{A}((\mathbb{G}, g, p), h)$ return 1 if $y = x$ return 0
--

Figure 3: Discrete Logarithm game.

ONE-MORE DISCRETE LOGARITHM PROBLEM. First formalised by Bellare et al. [2], this assumption allows access to a discrete logarithm oracle $\mathcal{O}_{\text{DL}}$ for as many as t queries and then asks for “one more”, the $(t + 1)$ -th.

Assumption 7 (One-More Discrete Logarithm [2]). *Let GrGen be a group generation algorithm, and let game $\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{OMDL}}$ be as defined in Figure 4. We say that the t -One-More Discrete logarithm (OMDL_t) problem is hard if, for any PPT adversary $\mathcal{A}$ playing $\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{OMDL}}$, it holds*

$$\text{Adv}_{\mathcal{A}, \text{GrGen}}^{\text{OMDL}}(t, \lambda) := \mathbb{P} \left[\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{OMDL}}(t, \lambda) = 1 \right] = \text{negl}(\lambda) \quad (5)$$

The security of OMDL in the GGM is a recent result. The first formal proof of this result was by Coretti et al. [5], later amended and corrected by Bauer et al. in [6]

Lemma 8 ([6]). *Under the generic group model, for any adversary $\mathcal{A}$ playing $\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{OMDL}}$, it holds*

$$\text{Adv}_{\mathcal{A}, \text{GrGen}}^{\text{OMDL}}(t, \lambda) \leq \text{negl}(\lambda)$$

4 The Algebraic One-More Discrete Logarithm assumption

GGM vs AGM. The *algebraic group model* (AGM) is close to the GGM, but exploits no group computation oracle $\mathcal{O}_{\mathbb{G} \times}$. Instead, it captures the idea that the adversary $\mathcal{A}$ knows how it used all received values $g_1, \dots, g_t$ to compute its current output h . Whenever $\mathcal{A}$ outputs a group element h , it is required to provide its representation with respect to all previous inputs, i.e. a vector $(\alpha_1, \dots, \alpha_t)$ such that

$$h = g_1^{\alpha_1} \dots g_t^{\alpha_t}$$

ALGEBRAIC ONE-MORE DISCRETE LOGARITHM. The Algebraic One-More Discrete Logarithm can be seen as a variant of OMDL where the adversary $\mathcal{A}$ behaves algebraically. It was introduced in [7] by Nick et al. as security assumption for the Schnorr-based multisignature scheme MuSig2.

Assumption 9 (One-More Discrete Logarithm [2]). *Let GrGen be a group generation algorithm, and let game $\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{AOMDL}}$ be as defined in Figure 4. We say that the Algebraic t -One-More Discrete logarithm (AOMDL_t) problem is hard if, for any PPT adversary $\mathcal{A}$ playing $\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{AOMDL}}$, it holds*

$$\text{Adv}_{\mathcal{A}, \text{GrGen}}^{\text{AOMDL}}(t, \lambda) := \mathbb{P} \left[\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{AOMDL}}(t, \lambda) = 1 \right] = \text{negl}(\lambda) \quad (6)$$

$\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{OMDL}}(t, \lambda)$	$\mathcal{O}^{\text{dl}}(h)$	$\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{AOMDL}}(t, \lambda)$	$\mathcal{O}^{\text{dl}}(h, \alpha, (\beta_i)_{i=0, \dots, t})$
$(\mathbb{G}, g, p) \leftarrow \text{GrGen}(1^\lambda)$	if $q = t$ return $\perp$	$(\mathbb{G}, g, p) \leftarrow \text{GrGen}(1^\lambda)$	if $q = t$ return $\perp$
$Q \leftarrow \emptyset$	$x \leftarrow \text{dlog}(h)$	$Q \leftarrow \emptyset$	$x \leftarrow \alpha + \sum_{i=0}^t x_i \beta_i$
$q \leftarrow 0$	$q \leftarrow q + 1$	$q \leftarrow 0$	$q \leftarrow q + 1$
for $i = 1, \dots, t + 1$	return x	for $i = 1, \dots, t + 1$	return x
$x_i \leftarrow \$\mathbb{Z}_p, h_i \leftarrow g^{x_i}$		$x_i \leftarrow \$\mathbb{Z}_p, h_i \leftarrow g^{x_i}$	
$Q[h_i] \leftarrow x_i$		$Q[h_i] \leftarrow x_i$	
$\vec{x} \leftarrow (x_1, \dots, x_{t+1})$		$\vec{x} \leftarrow (x_1, \dots, x_{t+1})$	
$\vec{h} \leftarrow (h_1, \dots, h_{t+1})$		$\vec{h} \leftarrow (h_1, \dots, h_{t+1})$	
$\vec{y} \leftarrow \mathcal{A}^{\mathcal{O}^{\text{dl}}}((\mathbb{G}, g, p), \vec{h})$		$\vec{y} \leftarrow \mathcal{A}^{\mathcal{O}^{\text{dl}}}((\mathbb{G}, g, p), \vec{h})$	
return 1 if $\vec{y} = \vec{x}$		return 1 if $\vec{y} = \vec{x}$	
return 0		return 0	

Figure 4: OMDL and AOMDL games. The main differences are highlighted in grey.

Lemma 10. *If OMDL_t is hard, then AOMDL_t is also hard.*

Proof. Consider a PPT adversary $\mathcal{A}$ playing $\text{Game}_{\text{GrGen}}^{\text{AOMDL}}$ for a group generation algorithm GrGen and winning with non-negligible probability. We can exhibit a PPT adversary $\mathcal{B}$ for $\text{Game}_{\text{GrGen}}^{\text{OMDL}}$ (and the same group generation algorithm GrGen) also winning with non-negligible probability by simply dropping the algebraic representation from all oracle queries. $\square$

FALSIFIABILITY. In [8], Gentry and Wichs introduce the notion of *falsifiability* for a cryptographic security assumption. Their goal is to capture the confidence we have in a problem, with non-falsifiability meaning that the problem is hard to reason about.

Definition 11 (Falsifiability [8]). *A cryptographic assumption that can be modeled as an interactive game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ is falsifiable if, for any adversary $\mathcal{A}$, the challenger $\mathcal{C}$ can decide in PPT if $\mathcal{A}$ won the game.*

Falsifiable assumptions are e.g. RSA, CDH, DDH, LWE, SVP, DL, factoring... In the AOMDL game, an adversary $\mathcal{A}$ only outputs group elements while interacting with the discrete logarithm

oracle $\mathcal{O}_{\text{DL}}$. Therefore the main difference between OMDL and AOMDL (see Figure 4) is that, whenever the adversary $\mathcal{A}$ queries $\mathcal{O}_{\text{DL}}$ on an element x , $\mathcal{A}$ must include an algebraic representation of x , i.e. a list of exponents $(\alpha, (\beta_i)_{i=1,\dots,t})$. With this we can implement the random oracle $\mathcal{O}_{\text{DL}}$ in PPT, making AOMDL falsifiable.

Lemma 12. *The claim that AOMDL_t is hard is a falsifiable cryptographic assumption.*

Sketch of proof. The game $\text{Game}_{\mathcal{A}, \text{GrGen}}^{\text{AOMDL}}$ of Figure 4 requires $\mathcal{A}$ to provide an algebraic representation $(\alpha, (\beta_i)_{i=1,\dots,t})$ of all queried h relative to the generator g and the challenges $h_1, \dots, h_{t+1}$. This allows the oracle $\mathcal{O}_{\text{DL}}$ to answer in PPT by returning $\alpha + \sum_i x_i \beta_i$, instead of $\text{dlog}(h)$, which could only be implemented in PPT if $\mathcal{A}$ could already break the DL assumption. $\square$

5 Use cases

Many protocols consider one-more security assumptions: blind signatures, multisignatures, threshold signatures, identification protocols, etc. We will focus on blind and threshold signatures.

BLIND SIGNATURES: CHAUM. Blind signatures allow users to obtain a signature of a message m from a signing party without it learning m . Bellare et al. [2] introduce One-More RSA to prove the security of Chaum's blind variant of the FDH-RSA signature protocol.

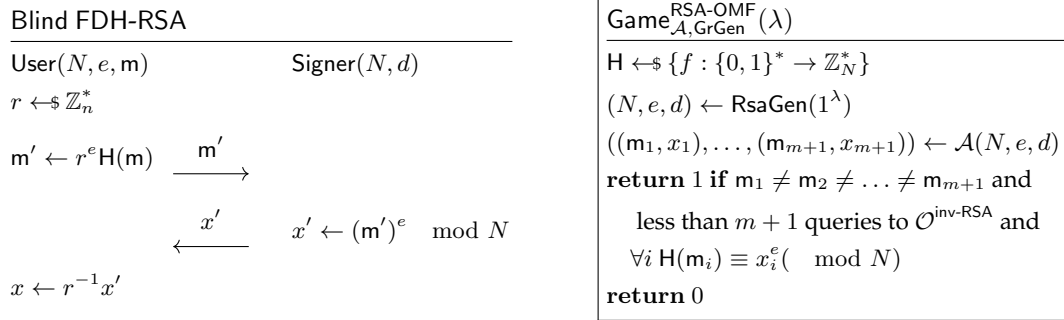


Figure 5: On the left, Chaum's blind FDH-RSA signature. On the right, its one-more forgery game.

Definition 13 (RSA Single-Target Inversion [2]). *Let RsaGen be the RSA parameter generation algorithm, and let game $\text{Game}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-OMF}}$ be as defined in Figure 5. We say that the blind FDH-RSA signature is one-more forgery secure if, for any PPT adversary $\mathcal{A}$ playing $\text{Game}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-OMF}}$, it holds*

$$\text{Adv}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-OMF}}(\lambda) := \mathbb{P} \left[\text{Game}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-OMF}}(\lambda) = 1 \right] = \text{negl}(\lambda) \quad (7)$$

Theorem 14 ([2]). *For any forger $\mathcal{F}$ attacking the RSA-OMF security of the FDH-RSA blind signature scheme in Figure 5, there exists a PPT adversary $\mathcal{A}$ against the $\text{RSA-ACTI}_{n,m}$ assumption such that*

$$\text{Adv}_{\mathcal{F}, \text{RsaGen}}^{\text{RSA-OMF}}(\lambda) \leq \text{Adv}_{\mathcal{A}, \text{RsaGen}}^{\text{RSA-ACTI}}(n, m, \lambda) \quad (8)$$

Proof. We will read [2] for a full description of the proof. $\square$

THRESHOLD SIGNATURES: SPARKLE. Threshold signature schemes allow to distribute the signing authority among n parties so that only subsets of (at least) t of them can sign messages m . The security of the threshold Schnorr variant Sparkle, by Crites et al. [9], is based on the AOMDL assumption. We refer interested readers to [9].

Theorem 15 ([9]). *For any forger $\mathcal{F}$ attacking the adp-TS-EUF-CMA security of Sparkle threshold signature scheme, there exists a PPT adversary $\mathcal{A}$ against the AOMDL_t assumption such that*

$$\text{Adv}_{\mathcal{F}, \text{GrGen}}^{\text{adp-TS-EUF-CMA}}(\lambda, \tau = 1) \leq \text{Adv}_{\mathcal{A}, \text{GrGen}}^{t\text{-AOMDL}}(\lambda) + \text{negl}(\lambda) \quad (9)$$

Sources and research material

This work is based on the following publications:

- [1] *First formalisation of One-More forgeries.* David Pointcheval and Jacques Stern. *Provably secure blind signature schemes.* 1996.
- [2] *First formalisation of OMRSAl and OMDL.* Bellare, Namprempre, Pointcheval, and Semanko. *The one-more-RSA-inversion problems and the security of Chaum’s blind signature scheme.* 2003.
- [3] *First use of the Discrete Logarithm problem.* Whitfield Diffie and Martin Hellman. *New Directions in cryptography.* 1976.
- [4] *First formalisation of the Discrete Logarithm problem.* Taher ElGamal. *A public key cryptosystem and a signature scheme based on discrete logarithms.* 1985.
- [5] *Sketch of proof of the security of OMDL in the GGM.* Sandro Coretti, Yevgeniy Dodis, and Siyao Guo. *Non-Uniform Bounds in the Random-Permutation, Ideal-Cipher, and Generic-Group Models.* 2018.
- [6] *Amended and corrected the proof in [5].* Balthazar Bauer, Georg Fuchsbauer, and Antoine Plouviez. *The One-More Discrete Logarithm Assumption in the Generic Group Model.* 2021.
- [7] *Introduction of AOMDL.* Jonas Nick, Tim Ruffing, and Yannick Seurin. *MuSig2: Simple two-round Schnorr multi-signatures.* 2021.
- [8] *Definition of falsifiable problem.* Craig Gentry and Daniel Wichs. *Separating succinct non-interactive arguments from all falsifiable assumptions.* 2011.
- [9] *The security of this threshold signature is based on the AOMDL assumption.* Elizabeth Crites, Chelsea Komlo, and Mary Maller. *Fully Adaptive Schnorr Threshold Signatures.*